Cryptographie R309
César, Bob et Alice

7 septembre 2026

Le but de cette séance est de présenter une méthode élémentaire de cryptage : le chiffre de César ou chiffrement par décalage. On implémentera notamment un algorithme de chiffrement, de déchiffrement et enfin un algorithme de décryptage capable de décoder le message sans la possession de la clé. Une petite partie théorique consistera à réviser la notion de calcul modulaire (utile pour l'implémentation) avec une petite application pratique classique d'abord (et un peu moins ensuite).

Rendu `cesar.py`

Le code devra être réalisé sous `Python`, les fonctions demandées devront être sauvegardées dans un fichier nommé `cesar.py`. Le fichier devra être versé sur le Moodle dans la boîte de dépôt prévue. Attention, le fichier rendu ne doit pas contenir de variables globales (sauf explicitement indiqué) ni même de tests. Il ne doit contenir que les fonctions demandées (en rose dans l'énoncé). Il est absolument nécessaire de respecter les noms des fonctions ou variables si spécifiés.

Test `test-cesar.py`

Un script de test est fourni afin que chaque étudiant puisse valider son code au fur et à mesure du TP.

Exercice 1 Calcul modulaire

1. Quelles sont parmi les égalités suivantes celles qui sont correctes ?

$$1 = 4[3] \tag{1}$$

$$-1 = 5[3] \tag{2}$$

$$7 = 1[3] \tag{3}$$

$$-1 = 1[3] \tag{4}$$

$$14 = 1[3] \tag{5}$$

2. Simplifier 10[9]. Faire de même pour 100[9]. D'une manière générale, simplifier $10^n[9]$ pour tout n entier naturel.

3. Soit x un entier naturel dont l'écriture décimale est de la forme

$$x = \overline{abcdef}$$

Montrer que si $a + b + c + d + e + f$ est un multiple de 9, alors il en est de même pour x.

4. Sans calculatrice, déterminer le(s)quel(s) de ces nombres est/sont des multiples de 9 : 126 ; 344 ; 243.

5. *(subsidiaire)*
   a) Montrer que si
   $$a - b + c - d + e - f$$
   est un multiple de 11, alors il en est de même pour x.
   b) Parmi les nombres qui suivent, lesquels sont divisibles par 11 ? 378 ; 198 ; 13574 ; 7232.

Exercice 2 Chiffre de César

Contexte. Bob et Alice, s'ennuyant profondément en classe s'échangent des petits messages afin de passer le temps. Cependant, leurs messages transitent de main en main dans leur salle de cours et ils ne souhaitent pas que leurs camarades de classe (ou pire leur professeur) prennent connaissance du contenu de ceux-ci. Ils décident donc de coder leurs messages à l'aide d'un procédé utilisé par César (l'empereur romain) en son temps : la substitution circulaire.

L'idée est assez élémentaire. Elle consiste à décaler chaque lettre du message d'un certain rang. Pour simplifier, ils n'utilisent que les lettres majuscules de l'alphabet et suppriment toute ponctuation ainsi que les espaces. Le rang choisi (c'est-à-dire le nombre de décalages) est la clé (ils comptent en changer régulièrement persuadés que cela permettra d'assurer la sécurité de leurs échanges).

Chiffrement. Pour leurs échanges du jour, ils ont choisi une clé égale à 3. Bob souhaite envoyer à Alice le message

```
Tu fais quoi ce soir ? Pizza ?
```

Dans un premier temps, il supprime tous les espaces, la ponctuation et transforme toutes les lettres en majuscules.

```
TUFAISQUOICESOIRPIZZA
```

Ensuite, il décale chaque lettre de 3 caractères dans l'alphabet. Dans le cas où il arriverait au bout de l'alphabet (i.e. après la lettre "Z"), il revient au début (i.e. la lettre "A"). Avec cette technique, il lui suffit donc de substituer les lettres selon le tableau suivant

Lettre	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Code	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Le message une fois codé, devient donc

```
WXIDLVTXRLFHVRLUSLCCD
```

On remarquera que les deux lettres Z du message sont codées par des C : c'est le « retour au début » de l'alphabet, qui se traduira dans le code par un calcul modulo 26. Il griffonne cela sur un bout de papier, puis l'envoie à Alice. Pour le déchiffrer, il suffit à Alice, qui connaît la clé, d'appliquer l'opération réciproque, c'est-à-dire de décaler les lettres de -3 rangs.

1. *(Mise en forme)* Afin d'aider Alice et Bob à communiquer, on souhaite définir des procédures de chiffrement et déchiffrement. Avant toute chose, il est nécessaire d'appliquer un prétraitement afin de s'assurer que le message ne contient que des lettres appartenant à l'alphabet utilisé. Avant le chiffrement, les accents du message seront supprimés (i.e. les é transformés en e), les lettres minuscules seront transformées en lettres majuscules et tout caractère n'appartenant pas à l'alphabet sera supprimé (notamment les espaces et la ponctuation). Définir une fonction `clean_message` qui réalise ce prétraitement.

2. *(Chiffrement)* Écrire une fonction `cesar` qui prend en argument un message ainsi qu'une clé et renvoie le message codé après avoir appliqué le prétraitement.

La réponse d'Alice au message de Bob est

```
MHYDLVWUDYDLOOHUPRQFRXUVGHFUBSWRFHVWWURSJHQLDO
```

Malheureusement, le message est intercepté par l'enseignant (fan de cryptographie à ses heures).

3. *(Déchiffrement)* Déchiffrer le message d'Alice à l'aide de la fonction `cesar` développée à la question précédente. Même s'ils reçoivent quelques remontrances, ils ne sont pas sanctionnés. Pourquoi ?

Alice et Bob se demandent toutefois comment leur enseignant a pu décoder leur message. Dans le cas du chiffre de César, la méthode la plus simple est la « force brute », qui consiste à tester toutes les clés possibles. Il n'y en a que 26 après tout. On peut donc toutes les essayer et obtenir ainsi 26 messages, dont l'un devrait être intelligible (la clé 0 redonnant bien sûr le message chiffré lui-même).

4. *(Force brute)* L'enseignant décide de fouiller les poubelles de la salle de classe et découvre d'anciens messages (utilisant une autre clé donc) échangés entre les deux comparses. Notamment le message suivant

   `FSFOWOXDVOCFKMKXMOC`

   Écrire une fonction `brute_force` qui prend en argument un message chiffré et renvoie la **liste des 26 déchiffrements possibles, indexée par la clé** : la case d'indice k contient le message obtenu en supposant que la clé utilisée était k. La case d'indice 0 contient donc le message chiffré lui-même. Par exemple,

   ```
   brute_force("WXIDLVTXRLFHVRLUSLCCD")[3]
   ```

   doit renvoyer `"TUFAISQUOICESOIRPIZZA"`.
   Afficher ensuite les 26 messages obtenus à partir du message trouvé dans les poubelles. Quelle est la clé utilisée ici ?

La lecture des 26 messages possibles reste assez laborieuse (notamment à cause de l'absence d'espacement et de ponctuation). Afin d'automatiser totalement le processus, on peut appliquer une analyse fréquentielle. Dans un langage donné, toutes les lettres n'apparaissent pas à la même fréquence. Comme on le voit sur la figure 1, certaines lettres sont nettement moins utilisées que d'autres. La lettre Z représente seulement 0.17% des lettres utilisées dans Wikipédia français, contre 16.39% pour la lettre E.

Afin de trouver automatiquement la valeur de la clé on peut, pour chacun des 26 décalages possibles, comparer la fréquence d'apparition des lettres du

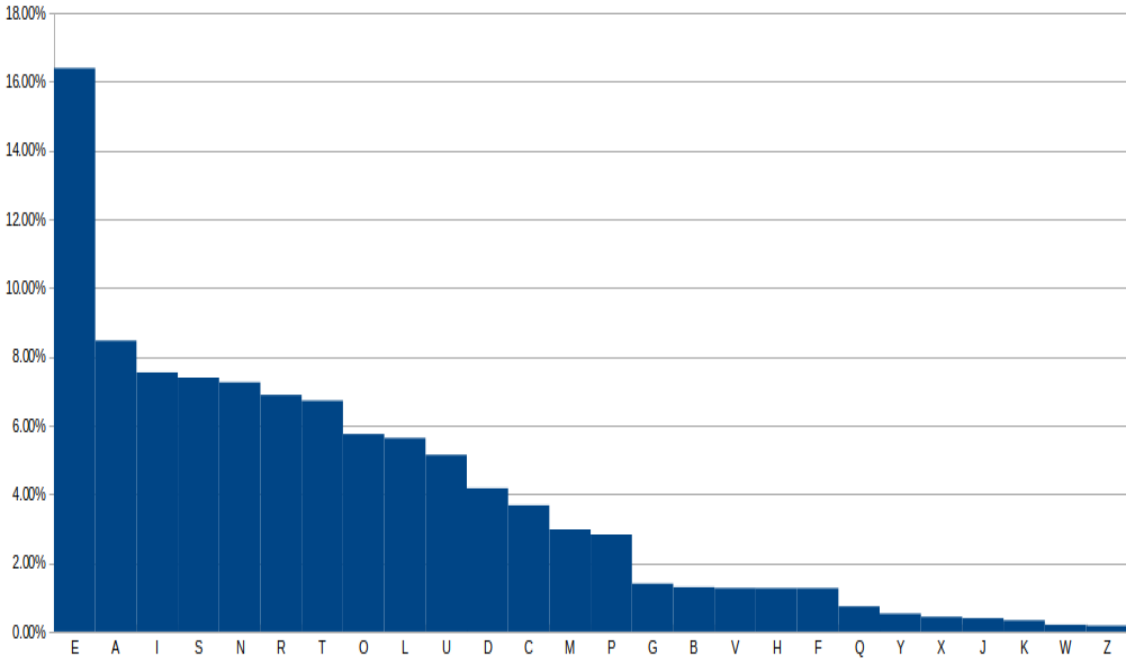


FIGURE 1 – Fréquence d'apparition des lettres en français (source Wikipédia)

message décodé avec les fréquences de référence en français. À cet effet, on pourra utiliser la fonction coût suivante

$$J(m) := \sum_{x \in \mathcal{A}} |F(m,x) - \overline{F}(x)|, \tag{6}$$

où x parcourt les lettres de l'alphabet $\mathcal{A}$, $F(m,x)$ est la fréquence d'apparition de la lettre x dans le message décodé m et $\overline{F}(x)$ sa fréquence dans le corpus de référence. Les valeurs d'apparition des lettres en français peuvent être trouvées dans le fichier `frequences_francais.txt`.

5. *(Calcul des fréquences)* Écrire une fonction `F` qui prend en argument un message et renvoie le tableau des fréquences d'apparition des lettres. Ces fréquences sont des *proportions*, comprises entre 0 et 1, dont la somme vaut 1. Par exemple,

   ```
   F("AAABBAAAZZ")
   ```

   doit renvoyer le tableau de taille 26

   ```
   [0.6, 0.2, 0, 0, ..., 0, 0.2]
   ```

6. *(Fréquences de référence)* Écrire une fonction `charge_frequences` qui lit le fichier `frequences_francais.txt` et renvoie un tableau contenant la fréquence d'apparition de chaque lettre dans le corpus de référence considéré. On veillera à ce que ces fréquences soient exprimées dans la même unité que celles renvoyées par `F`, c'est-à-dire sous forme de proportions comprises entre 0 et 1 (et non de pourcentages) : sans cela, la comparaison effectuée par (6) n'aurait aucun sens. Le tableau des fréquences sera stocké dans une variable globale `Frequences_ref` par

   ```
   Frequences_ref=charge_frequences()  # charge les frequences de reference
   ```

7. *(Fonction coût)* Définir une fonction `J` qui prend en argument un message m et renvoie la valeur de $J(m)$ définie par (6).

8. *(Décryptage)* Écrire une fonction `break_cesar` qui prend en argument un message codé et
   - calcule pour chaque décalage possible l'écart entre les fréquences d'apparition des lettres par rapport au corpus de référence à l'aide de la formule (6) ;

— détermine le décalage pour lequel l'écart est le plus faible ;

— renvoie la clé obtenue ainsi que le message décodé à l'aide de cette clé.

Quelques méthodes utiles en Python. `upper` ; `isalpha` ; `unidecode.unidecode` ; `ord` ; `chr` ;